1 Q1 placeholder

2 Q2 placeholder

3 Q3 placeholder

4 Q4: R-GCN Implementation

```python
class MyRGCNConv(MessagePassing):
    """R-GCN: one weight matrix per relation, plus a self-connection."""

    def __init__(self, in_channels, out_channels, num_relations):
        # aggr="mean" IS the 1/c_{i,r} of the formula, taking
        # c_{i,r} = |N_r(i)|: the example Schlichtkrull et al.'s Eq. (2)
        # itself offers. PyG's mean is 0, not NaN, when |N_r(i)| = 0, so
        # no empty neighbourhood needs a special case.
        super().__init__(aggr="mean")
        self.in_channels = in_channels
        self.out_channels = out_channels
        self.num_relations = num_relations
        self.root = nn.Linear(in_channels, out_channels, bias=True)  # W_0
        # W_r: one full [in, out] matrix per relation. Basic R-GCN, so
        # neither basis nor block-diagonal decomposition.
        self.rel = nn.ModuleList([
            nn.Linear(in_channels, out_channels, bias=False)
            for _ in range(num_relations)])
        self.reset_parameters()

    def reset_parameters(self):
        # Safe to call twice, and the harness does call it twice.
        super().reset_parameters()
        self.root.reset_parameters()
        for lin in self.rel:
            lin.reset_parameters()

    def forward(self, x, edge_index, edge_type):
        out = self.root(x)                                  # W_0 h_i
        n = x.size(self.node_dim)
        for r in range(self.num_relations):
            ei_r = edge_index[:, edge_type == r]            # relation r only
            if ei_r.size(1) == 0:
                continue
            # c_{i,r} does not depend on j and W_r is linear, so
            # sum_j (1/c_{i,r}) W_r h_j == W_r (mean_j h_j): transform
            # first, propagate second. size=(n, n) keeps the result at the
            # full node count where relation r touches only some nodes.
            out = out + self.propagate(ei_r, x=self.rel[r](x), size=(n, n))
        return out

    def message(self, x_j):
        return x_j
```

`self.rel[r]` is W_r: one full in $\times$ out matrix per relation, with neither of the decompositions Schlichtkrull et al. add as regularisers — the *basic* version. `self.root` is W_0, applied to h_i outside any propagation.

I took $c_{i,r} = |N_r(i)|$, the example their Eq. (2) offers, which is exactly `aggr="mean"`. Guarding it is not optional: all 15,466 nodes have $|N_r(i)| = 0$ for some r; PyG's mean divides by `count.clamp(min=1)`, returning 0 rather than NaN.

`edge_type` never reaches `propagate`: `forward` slices `edge_index` by relation and propagates once per subset, so relation r's edges only meet W_r and the stub's `message(self, x_j)` is untouched. W_r is applied before propagating rather than after — $c_{i,r}$ does not depend on j and W_r is linear, so $\sum_j c_{i,r}^{-1} W_r h_j = W_r(\text{mean}_j\, h_j)$ — transforming 15,466 rows instead of 3,014,361 messages. `size=(n, n)` keeps every relation's output at the full node count.

Cell 28's `train_and_eval_hetero("R-GCN")`, run unmodified, selected epoch 13:

	MRR	Hit@1
validation	0.2291	0.0782
test	0.2229	0.0714

Random tie-breaking over 21 candidates already scores MRR 0.174, and training loss settles at $\ln 2$ with the scores collapsed towards zero, so the model barely beats chance. Cell 26's features come from gensim's `Word2Vec` at `workers=4`, not reproducible across processes; over five runs validation MRR was 0.190 ± 0.020 and test MRR 0.182 ± 0.023, so read the table with that spread. The layer is not the cause: `torch_geometric.nn.RGCNConv` in the same harness scores 0.208, and my `forward` matches it to 4×10^{-7} on all 6,067,650 edges. The normalisation is the lever: pooling, as the paper's link-prediction experiments do ($c_{i,r} = c_i = \sum_r |N_r(i)|$), reaches 0.305 — not by smoothing the dense relations less, whose scale it barely changes ($\times 1.00$, $\times 0.98$), but by dividing the sparse `friend` term, whose edges *are* the training positives, by 22. Pooling `friend` alone gives 0.325.

5 Q5 placeholder